部署到 Render｜方法與步驟

> 適用版本：2026-10-16d 部署方式：Render 從 GitHub 讀取 repo 根目錄的 `Dockerfile`，自動建置和部署；之後每次推送到 `main` 都會自動重新部署 另一種部署方式（Firebase）請見 DEPLOY_FIREBASE.md

---

1. 架構

Render 上只跑**一個 Docker 容器**，同時提供網頁和後端 API。資料仍然存放在原本的 Firebase，所以 Render 版和 https://mycontractapp-911.web.app **共用同一批合約**。

元件	位置	說明
網站 + `/api/*`	Render Web Service（Docker）	`firebase/render/server.py`（Flask + Gunicorn）直接呼叫原本的 `functions/main.py`，後端邏輯和 Firebase 版完全相同
資料庫、檔案、登入	Firebase（Firestore、Storage、匿名登入）	透過「服務帳戶金鑰」連線
健康檢查	`/healthz`	回報金鑰、簽署金鑰、SMTP 是否設定齊全，不會顯示任何密碼

repo 裡和 Render 有關的檔案（兩個 repo 都有，內容相同）

檔案	用途
`Dockerfile`	Python 3.12-slim，安裝依賴、複製程式和網頁，用 Gunicorn 啟動；以非 root 使用者執行，內建健康檢查
`.dockerignore`	排除 `.env`、`venv`、測試和文件，金鑰不會被打包進映像檔
`render.yaml`	選用的 Render Blueprint，可以一次建好服務和環境變數欄位
`firebase/render/server.py`	Render 用的網頁伺服器
`firebase/render/requirements.txt`	Render 額外需要的套件（Gunicorn）

> 已驗證：在 Docker 中 build 映像檔成功（411 MB），容器連上 Firebase 模擬器跑完整 API 測試 **78/78 通過**，包含建立合約、上傳、翻譯、封面、兩方／三方簽署、PDF 和完成通知信。

---

2. 部署前準備

2.1 取得 Firebase 服務帳戶金鑰（必要）

1. 打開 Firebase 主控台 → 專案設定 → 服務帳戶。
2. 按 **「產生新的私密金鑰」** → 確認，會下載一個 `.json` 檔。
3. 這個檔案等同於資料庫的完整權限，**請勿放進 GitHub、Email 或聊天軟體**。

2.2 讓金鑰也能使用翻譯（英文翻譯才需要）

1. 確認已 啟用 Cloud Translation API。
2. 打開 IAM 權限，找到 `firebase-adminsdk-…@mycontractapp-911.iam.gserviceaccount.com`。
3. 按鉛筆圖示 →「新增其他角色」→ 選 **Cloud Translation API 使用者（Cloud Translation API User）** → 儲存。

繁簡互轉不需要這個權限。

2.3 找出簽署金鑰（必要）

打開 `C:\Users\USER\iCloudDrive\MyContractApp-eSign-main\firebase\functions\.env`，複製 `SIGNING_TOKEN_ENCRYPTION_KEY=` 後面的值。

> Render 版**必須使用和 Firebase 版相同的金鑰**。兩邊共用同一個資料庫，金鑰不同的話，另一邊發出的連結就無法提醒或下載。

2.4 寄信方式（重要）

Render 免費方案封鎖 SMTP 的 25、465、587 連接埠（Render 公告，2025-09 起），所以 Gmail 的 `smtp.gmail.com:587` 在免費方案上**寄不出去**。請從下面三種方式選一種：

方式	設定	適合
A. 付費方案（Starter 起）	照常使用 Gmail：`SMTP_HOST=smtp.gmail.com`、`SMTP_PORT=587`	正式使用（推薦）
B. 免費方案 + 支援 2525 埠的寄信服務	例如 Brevo、Mailgun、SendGrid 的 SMTP，`SMTP_PORT=2525`	測試或低流量
C. 不寄信	SMTP 留白	管理台會顯示「未寄信（手動轉交）」，需要自己複製連結轉給對方

---

3. 在 Render 建立服務（網頁操作）

方法一：Web Service（一般做法）

1. 登入 https://dashboard.render.com，右上角按 **New** → **Web Service**。
2. **Connect a repository**：第一次使用時按「Connect GitHub」並授權，然後選 `SamWang61/MyContractApp-eSign` 或 `SamWang61/E-ContractApp`（兩個都可以）。
3. 依下表填寫：

欄位	填寫內容
Name	`mycontractapp`，網址會是 `https://mycontractapp.onrender.com`；名稱被用過時，Render 會自動加上字尾
Language / Runtime	**Docker**
Branch	`main`
Region	**Singapore**（離台灣最近）
Root Directory	留白
Dockerfile Path	`./Dockerfile`（預設值）
Docker Build Context Directory	`.`（預設值）
Instance Type	**Starter** 以上（需要寄信或不想休眠時）；Free 也能跑，但見第 7 章

1. 展開 **Environment Variables**，依第 4 章加入變數。
2. 展開 **Advanced** → **Health Check Path** 填 `/healthz`；**Auto-Deploy** 選 `On Commit`。
3. 按 **Deploy Web Service**。第一次 build 約 3–6 分鐘，之後有快取會更快。

方法二：Blueprint（用 `render.yaml` 一次建好）

1. **New** → **Blueprint** → 選 repo → Render 會讀取 `render.yaml`。
2. 畫面會列出需要填寫的機密變數（標示為 `sync: false` 的項目），逐一填入第 4 章的值。
3. 按 **Apply**。

---

4. 環境變數

在服務的 **Environment** 頁面新增：

變數	必填	值
`FIREBASE_SERVICE_ACCOUNT_JSON`	是	第 2.1 節下載的 `.json` **整個檔案內容**（從 `{` 到 `}` 整段貼上），也可以貼 base64 編碼後的字串
`SIGNING_TOKEN_ENCRYPTION_KEY`	是	第 2.3 節複製的值，必須和 Firebase 版相同
`APP_BASE_URL`	建議	簽署連結和 Email 裡使用的網址，例如 `https://mycontractapp.onrender.com`。留白時會自動使用 Render 提供的網址
`GOOGLE_CLOUD_PROJECT`	否	預設 `mycontractapp-911`
`STORAGE_BUCKET`	否	預設 `mycontractapp-911.firebasestorage.app`
`SMTP_HOST`、`SMTP_PORT`、`SMTP_USERNAME`、`SMTP_PASSWORD`、`SMTP_FROM`	寄信時	見第 2.4 節
`COMPLETION_CC`	否	合約完成時的副本收件人，多個用逗號分隔
`WEB_CONCURRENCY`	否	Gunicorn 的 worker 數，預設 2；Free 方案（512 MB 記憶體）建議設為 1

不想把金鑰貼進環境變數的話，可以改用 Secret File：到 **Environment → Secret Files → Add Secret File**，檔名填 `firebase-service-account.json`，內容貼上金鑰 JSON。系統會自動讀取 `/etc/secrets/firebase-service-account.json`，這時 `FIREBASE_SERVICE_ACCOUNT_JSON` 就不用填。

修改環境變數後，Render 會自動重新部署。

---

5. 部署後設定與驗證

5.1 把 Render 網域加入 Firebase 授權網域

Authentication → 設定 → 授權網域 → **新增網域** → 填入 `mycontractapp.onrender.com`（換成你實際的 Render 網域；有自訂網域的話也一起加）。

5.2 驗證

檢查	方法	正確結果
設定是否齊全	打開 `https://<你的網域>/healthz`	`"ok": true`、`"credentials": "env:FIREBASE_SERVICE_ACCOUNT_JSON"`（或 `secret-file`）、`"signingKey": true`
後端版本	打開 `/api/health`	`"version": "2026-10-16d"`
網站	打開首頁、`/contracts`	與 Firebase 版相同
完整流程	建立兩方合約 → 發送邀請 → 用兩個連結各簽一次	狀態變成「已全數簽署」，可以下載 PDF；有設定寄信的話，會收到完成通知

`/healthz` 回傳 **503** 時，代表金鑰或簽署金鑰沒設定好。Render 的 **Logs** 會用中文寫明缺少哪一項。

6. 日常更新

想做的事	做法
更新程式	推送到 GitHub `main`，Render 會自動 build 並部署，舊版本會持續服務到新版本通過健康檢查
手動重新部署	服務頁面右上角 **Manual Deploy** → **Deploy latest commit**
回到上一版	**Events** 頁面 → 選之前成功的部署 → **Rollback**
查看日誌	**Logs** 頁面（Gunicorn 存取紀錄與錯誤訊息）
自訂網域	**Settings** → **Custom Domains** → **Add**，依指示在 DNS 加 CNAME；記得同步更新 `APP_BASE_URL`，並把網域加入 Firebase 授權網域

7. 注意事項

項目	說明
Free 方案休眠	閒置 15 分鐘後會休眠，下一次打開需要等約 30–60 秒才會醒來。簽署人點連結時可能會覺得很慢，正式使用建議選 Starter
Free 方案寄信	SMTP 25/465/587 被封鎖，見第 2.4 節
兩邊同時運作	Firebase 版和 Render 版可以同時上線。Email 裡的連結會指向**發出邀請那一邊**的 `APP_BASE_URL`，兩邊都能正常簽署
只想用 Render	可以停掉 Firebase Hosting，但 Firestore、Storage 和 Authentication 仍然要保留，資料都存在那裡
金鑰安全	服務帳戶金鑰只放在 Render 的環境變數或 Secret File。`.dockerignore` 已排除 `.env`，金鑰不會被打包進映像檔或推上 GitHub
上傳大小	單一檔案 10 MB，伺服器接收上限 25 MB（base64 編碼後的大小）

8. 常見問題

狀況	原因	解決方式
Build 失敗：`pip install` 錯誤	暫時性的網路問題	**Manual Deploy → Clear build cache & deploy**
部署卡在健康檢查失敗	`/healthz` 回 503，金鑰或簽署金鑰沒設定	看 Logs 裡的中文提示，補上環境變數
打開網站時跳出「尚未登入」	可能是 Render 網域沒有加入 Firebase 授權網域，或瀏覽器封鎖第三方 Cookie	照第 5.1 節加入網域，然後重新整理頁面
API 回 500，Logs 出現 `DefaultCredentialsError`	金鑰 JSON 貼得不完整	重新貼上**完整**的 JSON（從 `{` 到 `}`）
翻成英文失敗，Logs 出現 `403`	金鑰沒有翻譯權限	照第 2.2 節加入角色
寄信逾時（timeout）	Free 方案封鎖 SMTP	改用付費方案，或改用 2525 埠
舊合約的連結無法提醒或下載	和 Firebase 版的簽署金鑰不同	把 `SIGNING_TOKEN_ENCRYPTION_KEY` 改成和 Firebase 版一致
第一次打開很慢	Free 方案休眠中	等候約 1 分鐘，或升級方案

9. 在自己電腦上測試 Docker（選用）

```
# 在 repo 根目錄
docker build -t mycontractapp .
docker run --rm -p 10000:10000 \
  -e FIREBASE_SERVICE_ACCOUNT_JSON="$(cat 金鑰.json)" \
  -e SIGNING_TOKEN_ENCRYPTION_KEY=與 Firebase 相同的值 \
  -e APP_BASE_URL=http://localhost:10000 \
  mycontractapp
# 打開 http://localhost:10000/healthz
```

注意：本機測試連到的是**正式的 Firebase 資料庫**，建立的測試合約會出現在正式資料裡。